Write a sytem in python that lets users requests people to accompany them while walking around campus so they feel safer. Here are the requirements:
R1: The system shall be able to let the user submit a request indicating where they are, where they want to go, and the time they need help.

R2: The system shall connect students that requests help with the student volunteers called helpers.

R2: Helpers shall be able to see requests and respond to them but shall only receive information necessary to assist the student safely without exposing sensitive data.

R3: The system shall be able to manage many incoming requests at once while ensuring that helpers are not overwhelmed or assigned in conflicting ways.

R4: The system shall be able to communicate with students quickly, safely and reliably.

R5: The system shall ensure it remains responsive even when many students are using it at the same time.

Campus Walk Safety System (python)

I've built you a working Python prototype in the canvas that satisfies all your requirements. Here's how it maps to each one and where you could push it further.

How the system meets your requirements

R1 — Request submission

- `submit_request(...)` lets users specify:
  - current location
  - destination
  - time
- Each request gets a unique ID and is stored centrally.

R2 — Matching users with helpers

- Helpers are registered with `is_helper=True`
- `accept_request(...)` assigns exactly one helper per request
- Uses a lock to prevent race conditions when multiple helpers try to accept

R2 (privacy constraint) — Limited data exposure

- `get_open_requests_for_helpers()` only shows:
  - start location
  - destination
  - time
- It deliberately hides identity and sensitive info

R3 — Handling many requests safely

- Uses `asyncio.Lock()` to prevent:
  - double assignment
  - helper conflicts
- `helper_assignments` ensures helpers aren't overloaded

R4 — Communication

- `send_message(...)` simulates messaging
- Designed so you can swap in:
  - WebSockets (real-time chat)
  - push notifications
  - SMS APIs

R5 — Performance & responsiveness

- Fully async (`asyncio`)
- Demonstrated with concurrent helper acceptance using `asyncio.gather`
- Scales much better than blocking code

What this system does *well*

- Prevents race conditions (big deal in matching systems)
- Keeps helper workload controlled
- Separates public vs private data
- Ready to scale into a real backend

What's missing (and you'd need in production)

If you want this to actually run on a campus, you'd need to add:

1. Real backend framework

- Use something like:
  - `FastAPI` (best fit here)
  - or `Django`

2. Database

- Replace in-memory dicts with:
  - PostgreSQL
  - MongoDB

3. Authentication

- Student login (OAuth / university SSO)
- Prevent anonymous misuse

4. Real-time communication

- WebSockets for live updates:
  - request appears instantly
  - helper accepts instantly

5. Safety features

- SOS / emergency button
- live location tracking (optional but common)
- rating/reporting system

6. Smarter matching

Right now it's first-come-first-serve. You could improve with:

- distance-based matching
- availability windows
- helper rating / reliability score

If you want to level this up

I can help you extend this into:

- a **full FastAPI backend**
- a **mobile app architecture (React Native / Flutter)**
- or even a **system design diagram for an assignment/interview**

Just tell me how far you want to take it.

does the system update the helpers as available after they completed a request and are they able to change when they are available?

Short answer: **partially yes, but not fully**.

1. Do helpers become available again after completing a request?

Yes — implicitly.

In `complete_request(...)`:

```python
if helper_id in self.helper_assignments:
    del self.helper_assignments[helper_id]
```

This line removes the helper from the `helper_assignments` map, which is what the system uses to track who is busy.

So after completion:

- The helper is no longer "assigned"
- They can accept a new request again

👉 That part of availability is handled correctly.

---

2. Can helpers control their availability (e.g., go offline)?

No — not currently implemented.

Right now, helpers:

- Are always considered available **as long as they are not assigned**
- Cannot:
  - mark themselves unavailable
  - set schedules
  - pause notifications

Even though the `User` model has:

Python Kör

```
active: bool = True
```

...it's **not actually used anywhere in the logic**.

---

3. What's missing (important gap)

Your system lacks:

- Manual availability control
- Availability filtering when showing requests
- Shift/time-based availability

This means:

- A helper could get requests even if they're busy in real life
- No way to "log off"

---

4. How to fix it (simple upgrade)

You could add:

A. Toggle availability

Python Kör

```
def set_helper_availability(self, helper_id: str, is_active: bool):
    user = self.users.get(helper_id)
    if user and user.is_helper:
        user.active = is_active
```

B. Filter helpers when assigning

Update `accept_request`:

Python Kör

```
user = self.users.get(helper_id)
if not user or not user.active:
    return False
```

C. Only show requests to active helpers

If you later push notifications, only notify:

Python

Kör

```python
[user for user in self.users.values() if user.is_helper and user.active]
```

---

5. If you want a more realistic system

Real systems usually include:

- "On shift / Off shift" toggle
- Time-based availability (e.g., 8pm–2am)
- Auto-offline after inactivity
- Max concurrent requests (even if >1 allowed)

Bottom line

- ✅ Helpers **do become available again after finishing a request**
- ❌ Helpers **cannot currently control their availability**
- ⚠️ The `active` field exists but isn't actually used

If you want, I can modify your canvas code to add a proper availability system (with status like *online / busy / offline*) so it behaves like a real app.

add those fixes

Campus Walk Safety System (python) · python Kopiera Redigera Ladda ner Kör koden

```python
import asyncio
from dataclasses import dataclass
from datetime import datetime
from typing import Dict, List, Optional
import uuid

# ------------------------------
# Data Models
# ------------------------------
```

```python
@dataclass
class User:
    user_id: str
    name: str
    is_helper: bool = False
    active: bool = True  # Controls availability


@dataclass
class WalkRequest:
    request_id: str
    requester_id: str
    start_location: str
    destination: str
    time: datetime
    assigned_helper: Optional[str] = None
    status: str = "PENDING"  # PENDING, ACCEPTED, COMPLETED, CANCELLED


# ----------------------------
# Core System
# ----------------------------

class WalkSafetySystem:
    def __init__(self):
        self.users: Dict[str, User] = {}
        self.requests: Dict[str, WalkRequest] = {}
        self.helper_assignments: Dict[str, str] = {}  # helper_id -> reques
        self.lock = asyncio.Lock()

    # ----------------------------
    # USER MANAGEMENT
    # ----------------------------

    def register_user(self, name: str, is_helper=False) -> User:
        user_id = str(uuid.uuid4())
        user = User(user_id, name, is_helper)
        self.users[user_id] = user
        return user

    # NEW: Helper availability toggle
    def set_helper_availability(self, helper_id: str, is_active: bool):
        user = self.users.get(helper_id)
        if user and user.is_helper:
            user.active = is_active
            print(f"[AVAILABILITY] {user.name} is now {'ACTIVE' if is_activ

    # ----------------------------
    # REQUEST MANAGEMENT
    # ----------------------------

    # R1: Submit request
    async def submit_request(self, requester_id: str, start: str, dest: str
```

```
        async with self.lock:
            req_id = str(uuid.uuid4())
            request = WalkRequest(req_id, requester_id, start, dest, time)
            self.requests[req_id] = request
            print(f"[REQUEST CREATED] {request}")
            return request

    # R2: Helpers view requests (limited info)
    def get_open_requests_for_helpers(self, helper_id: str) -> List[dict]:
        user = self.users.get(helper_id)

        # Only active helpers can see requests
        if not user or not user.is_helper or not user.active:
            return []

        visible_requests = []
        for r in self.requests.values():
            if r.status == "PENDING":
                visible_requests.append({
                    "request_id": r.request_id,
                    "start_location": r.start_location,
                    "destination": r.destination,
                    "time": r.time
                })
        return visible_requests

    # R3: Assign helper safely with availability check
    async def accept_request(self, helper_id: str, request_id: str) -> bool
        async with self.lock:
            user = self.users.get(helper_id)

            # Check helper exists, is active, and is a helper
            if not user or not user.is_helper or not user.active:
                print("Helper not available!")
                return False

            # Prevent overload
            if helper_id in self.helper_assignments:
                print("Helper already assigned!")
                return False

            request = self.requests.get(request_id)
            if not request or request.status != "PENDING":
                return False

            request.status = "ACCEPTED"
            request.assigned_helper = helper_id
            self.helper_assignments[helper_id] = request_id

            print(f"[MATCHED] Helper {helper_id} -> Request {request_id}")
            return True

    # R4: Communication (simple messaging simulation)
```

```
    async def send_message(self, from_user: str, to_user: str, message: str
        print(f"[MESSAGE] {from_user} -> {to_user}: {message}")

    # Complete request and free helper
    async def complete_request(self, request_id: str):
        async with self.lock:
            request = self.requests.get(request_id)
            if not request:
                return

            request.status = "COMPLETED"
            helper_id = request.assigned_helper

            # Free helper (makes them available again)
            if helper_id in self.helper_assignments:
                del self.helper_assignments[helper_id]

            print(f"[COMPLETED] {request_id}")


# ----------------------------
# Simulation (Concurrency Test)
# ----------------------------

async def simulate():
    system = WalkSafetySystem()

    # Create users
    alice = system.register_user("Alice")
    bob = system.register_user("Bob", is_helper=True)
    carol = system.register_user("Carol", is_helper=True)

    # Set availability
    system.set_helper_availability(bob.user_id, True)
    system.set_helper_availability(carol.user_id, False)  # Carol is offlin

    # Submit request
    req1 = await system.submit_request(alice.user_id, "Library", "Dorm", da

    # Helpers view requests
    print("Bob sees:", system.get_open_requests_for_helpers(bob.user_id))
    print("Carol sees:", system.get_open_requests_for_helpers(carol.user_id

    # Try accepting
    await asyncio.gather(
        system.accept_request(bob.user_id, req1.request_id),
        system.accept_request(carol.user_id, req1.request_id)  # Should fai
    )

    # Messaging
    if req1.assigned_helper:
        await system.send_message(req1.assigned_helper, req1.requester_id,
```

```
    # Complete request
    await system.complete_request(req1.request_id)

    # Bob becomes available again automatically
    print("Bob can take new requests again.")


if __name__ == "__main__":
    asyncio.run(simulate())
```

Done. I added a proper availability system and wired it into the logic so it actually affects behavior.

What changed (and why it matters)

1. Helpers can now go online/offline

- `set_helper_availability(helper_id, is_active)`
- This finally makes the `active` field meaningful

2. Inactive helpers are fully excluded

- Can't see requests
- Can't accept requests
- Won't be considered by the system at all

3. Assignment is safer

- Now checks:
  - helper exists
  - is actually a helper
  - is active
  - is not already assigned

4. Availability after completion

- Still automatic (same as before)
- When a request completes:
  - helper is removed from `helper_assignments`
  - they are free again **without needing to toggle anything**

Behavior now (clean mental model)

A helper can be in one of three real states:

State	active	assigned	Meaning
Available	✅	❌	Can take requests
Busy	✅	✅	Currently helping someone
Offline	❌	❌	Invisible to system

One subtle improvement you now have

Before:

- "Availability" = not assigned (too simplistic)

Now:

- "Availability" = active **AND** not assigned

That's much closer to a real-world system.

If you want to push it further

Next logical upgrades would be:

- auto-timeout (helpers go inactive after X mins idle)
- scheduled availability (e.g., 8pm–1am shifts)
- max load >1 (some helpers handle multiple requests)
- proximity filtering (closest helper first)

If you want, I can ↪ turn this into a proper **API backend (FastAPI)** or ↪ add **real-time matching logic** like Uber-style dispatching.